

Interactive Real-time Leader Follower Control System for UAV and UGV

Aidana Tassanbi

School of Engineering and Digital Science
Nazarbayev University
Nur-Sultan, Kazakhstan
aidana.tassanbi@nu.edu.kz

Ton Duc Do

School of Engineering and Digital Science
Nazarbayev University
Nur-Sultan, Kazakhstan
doduc.ton@nu.edu.kz

Aziz Iskakov

School of Engineering and Digital Science
Nazarbayev University
Nur-Sultan, Kazakhstan
aziz.iskakov@nu.edu.kz

Md. Hazrat Ali*

School of Engineering and Digital Science
Nazarbayev University
Nur-Sultan, Kazakhstan
*md.ali@nu.edu.kz

Abstract — This article is aimed at creating a leader-follower system for Unmanned Aerial vehicles (UAV) and Unmanned Ground vehicles (UGV) with obstacle avoidance. In this system, the UAV acts as a leader, which is given the coordinates of a predetermined path, along which it must follow according to its algorithm. While the UGV is a follower, that needs to follow the leader by avoiding obstacles. QDrone and QBot from QUANSER were used as vehicles, and the OptiTack tracking system was applied for determining their locations. The article presents simple algorithms created for both vehicles and the results of the experiment. The experiment showed that all algorithms work flawlessly. However, problems arose with the accuracy of placing obstacles due to the limited power of the follower's onboard computer.

Keywords — Leader-Follower system, UAV, UGV, QUARC, OptiTack, obstacle avoidance.

I. INTRODUCTION

Today, process automation is an integral part of any advanced system. Automated auxiliary robots are popular not only in production but also in everyday life, for example, vacuum cleaner robots, waiter robots, etc. The demand for unmanned vehicles and other modes of transport is growing every year. Moreover, more and more research has recently been carried out aimed at creating leader-follower algorithms for multi-vehicle systems. These systems can be used in a wide range of applications, from military applications to delivery services.

A. Related work

The leader-follower system for unmanned vehicles may be in high demand. There are many studies that consider systems consisting of several Unmanned Ground Vehicles (UGVs) and Unmanned Aerial Vehicles (UAVs) which form a formation when following the leader [1-3]. Gopakumar and Shihabudheen considered the system with 1 leader and 6 followers, three of which are quadrotors, while the other 4 vehicles are two-wheeled mobile robots (2WMRs) [4]. This study aimed to develop a cooperative control algorithm for

reaching a consensus between the agents through communication. Other studies considered the loss of one of the agents (followers) and the ability of them to re-form into another formation during the mission [5-7]. Research has also been carried out to create a leader-follower system consisting of several 2WMRs that are able to bypass obstacles. In this case, the obstacle is detected by the leader, that re-computes the route, while the followers simply follow the leader [8]. In this study, the authors use the potential field method, the main idea of which is to determine the range of obstacles for different orientations of the sensors and to determine the polar density of the obstacle. After that, the path with the lowest density is determined, which the vehicle must follow in order to avoid the obstacle and reach the target point [8]. Apart from that, a system was created where the UAV follows the landing station in the form of a UGV to be able to land on it at any time [9]. This article aims to create a controller for a leader-follower system for a quadcopter and a two-wheeled mobile robot with obstacle avoidance. This system is different in that the leader is a drone that has no obstacles in the air. Whereas the robot must follow the drone while creating its way through obstacles. This system can be used in many areas. For example, in delivery services, in the study of hard-to-reach places and objects, military purposes, and rescue works. Indeed, in most cases, there are fewer obstacles in the air than on the ground, which creates the need to create such a system.

II. METHODS

A. Dynamic modeling

Using Newton's laws and the Euler-Lagrange method, you can create a dynamic model of a quadcopter. The motion of the UAV can be described using a system consisting of six non-linear differential equations:

$$\begin{cases} \ddot{x} = (\sin \psi + \cos \psi \sin \theta \cos \varphi) \frac{F_z}{m}, & \ddot{\phi} = \frac{F_{\phi}}{J_{xx}} \\ \ddot{y} = (\sin \psi \sin \theta \cos \varphi - \cos \psi \sin \theta) \frac{F_z}{m}, & \ddot{\theta} = \frac{F_{\theta}}{J_{yy}} \\ \ddot{z} = -g + (\cos \varphi \sin \theta) \frac{F_z}{m}, & \ddot{\psi} = \frac{F_{\psi}}{J_{zz}} \end{cases} \quad (1)$$

where x , y , and z are the Cartesian coordinates of the quadcopter (Fig. 1); φ , θ , ψ are the Euler angles (φ is the yaw, θ is the pitch, and ψ is the roll angle); J_{xx} , J_{yy} , and J_{zz} are diagonal elements of UAV's inertia tensor; m is the mass of the quadcopter; g is the acceleration due to gravity; $F = (F_z, F_\varphi, F_\theta, F_\psi)$ – virtual control forces, which can be described by the motor control forces as:

$$\begin{aligned} F_z &= T_1 + T_2 + T_3 + T_4 \\ F_\varphi &= L(T_3 - T_4) \\ F_\theta &= L(T_1 - T_2) \\ F_\psi &= C_y(T_1 + T_2 - T_3 - T_4) \end{aligned} \quad (2)$$

where L is the distance between UAV's center and its motor, C_y is a constant obtained experimentally, that depends on the relationship between thrust and yawing moment; T – thrust force generated by the rotation of propellers.

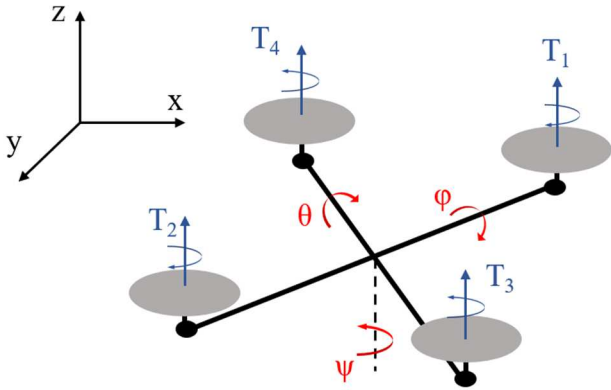


Fig. 1. Schematic diagram for UAV.

In this experiment UGV (QBot 2e) has only two wheels that are connected to the motor, that can rotate and act independently from each other. This allows QBot to drive and make a U-turn right on the spot around its axis. From Fig. 1 it can be seen that point C is set in the center of the wheel axis and this point can be counted as the current position of the robot with (x, y) coordinates and orientation angle φ that was taken with respect to the x -axis.

Since the floor surface is a rough, dense rubber with an abrasive pattern, a lot of friction is established between the QBot's wheels and the floor, allowing the robot to move and turn with negligible slip. It is also worth considering that the weight of the robot is not sufficient to deform the wheels. Considering all this, the non-linear cinematic equation for the Robot will be the following:

$$\dot{C} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\varphi} \end{bmatrix} = \begin{bmatrix} \cos \varphi & 0 \\ \sin \varphi & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix} \quad (3)$$

where v and ω are linear and angular velocities of QBot, respectively. Since Qbot's wheels are driven separately, we will have two equations for them, that are:

$$\begin{cases} v_R = v + l\omega \\ v_L = v - l\omega \end{cases} \quad (4)$$

where l is the distance from the center of the wheels' axis to each wheel. With simple manipulation, the equation for the angular velocity of each wheel can be calculated as shown below, where R corresponds to the radius of the wheels.

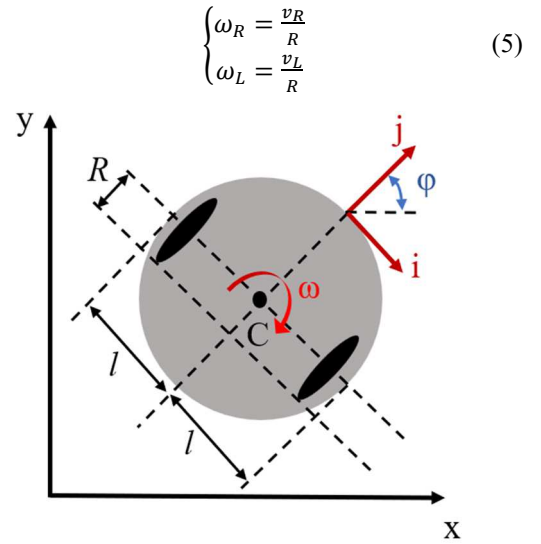


Fig. 2. Schematic diagram for UGV.

B. Communication and Localization

Communication between the vehicles and the computer is provided through a router and a wireless Wi-Fi connection, which assigns an individual IP address to the computer, drone, and robot. By setting up hardware settings in MATLAB Simulink and using QUARC blocks that can transmit information to the carrier of the specified IP address, control and command of vehicles were provided.

To determine the location of vehicles in space, the OptiTrack system was used. For this, 6 cameras were installed at the corners and sides of the workspace and connected to the ground station (PC) by wires. The cameras record the location of special markers that have been attached to vehicles and determine the location and orientation of an object. For each vehicle for identification, 5 such markers were attached according to a certain scheme. This system is quite accurate and, according to the manufacturer, allows determining the location of an object with an accuracy of 2 mm [10].

C. Leader-Follower algorithm

Communication with the drone and robot is first established when starting the system. As soon as the connection has been confirmed and all three Simulink models have been launched without problems, the "Mission server" receives, processes, and sends data from OptiTrack to the leader and follower. This data contains information about the location and orientation of the vehicles. If OptiTrack receives incorrect data or does not recognize the vehicles at all, the QDrone Commander model is immediately terminated. To run the algorithm, it is necessary for QDrone to take off to a height of at least 0.5 m, this is done using the joystick by giving the drone the "Arm" and "Takeoff" commands. As soon as both switches have been activated, QBot starts its algorithm which is provided in Fig. 3.

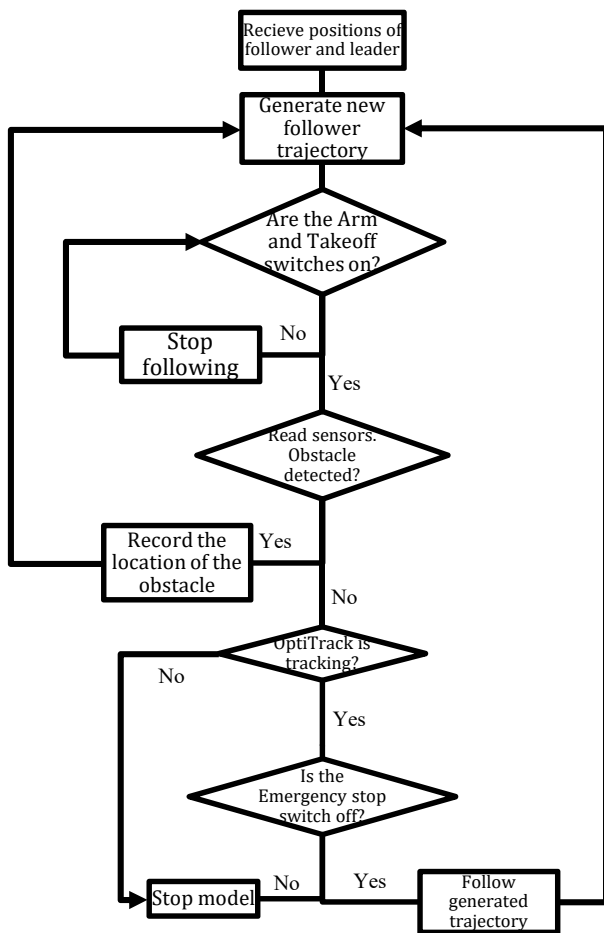


Fig. 3. Algorithm for the follower.

As soon as the QBot Commander receives the coordinates of the leader and follower, it generates the follower Trajectory. This data is transformed into angular and linear speed, which are given to the QBot. If an obstacle is detected by sensors, the robot will recalculate its trajectory using the obstacle avoidance algorithm described below. As can be seen from Fig. 3, the algorithm continues to run until a tracking issue occurs or the "Stop" button on the joystick is triggered. If the "Arm" or "Take off/Landing" switches are turned off, the robot will stop following the leader, but will continue to calculate the trajectory.

When the system is started, the QDrone hovers over the takeoff point, and the QBot approaches it until the distance between them is less than 0.25 m. As soon as the "Start/Stop" button is triggered in "Mission Server", the algorithm starts, according to which the QDrone flies to predetermined points. As can be seen from Fig. 4, the "Mission Server" constantly compares the coordinates of the leader and follower. As soon as the follower approaches the leader, the algorithm sends the next waypoint to the leader. When the system reaches the last waypoint, the "Mission Server" sends the coordinates of the center of the work area so that the drone will land in the center.

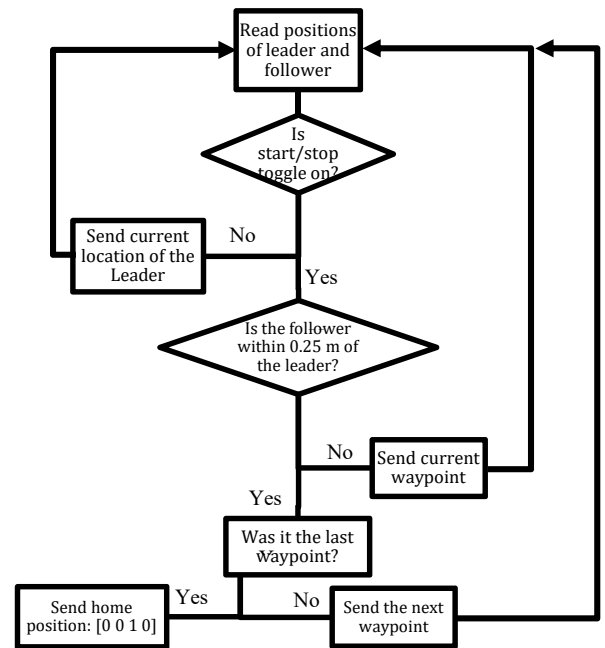


Fig. 4. Mission server algorithm.

With regard to UAV, its algorithm consists of checking the system for battery level, flight altitude, communication or localization problems, and receiving commands to land or follow to a new position. This algorithm is shown in Fig. 3. In addition, the QDrone, as the leader, constantly sends its location to the follower.

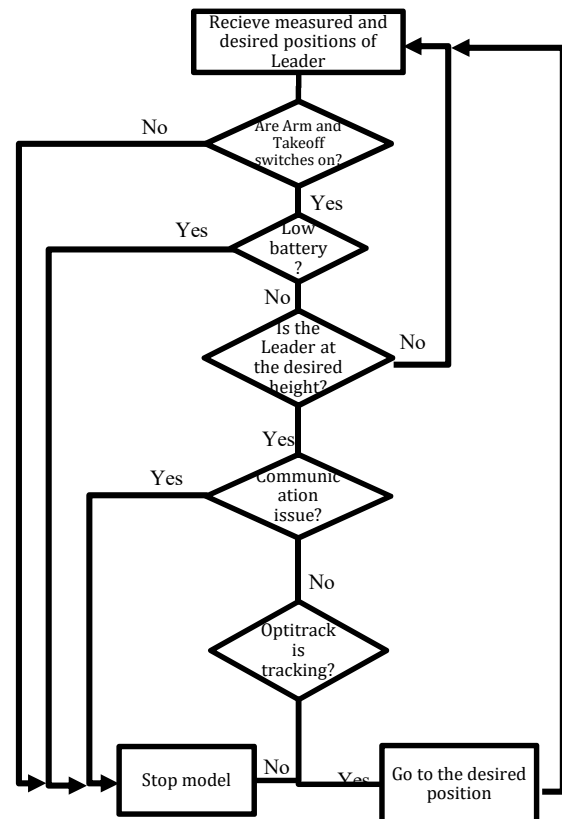
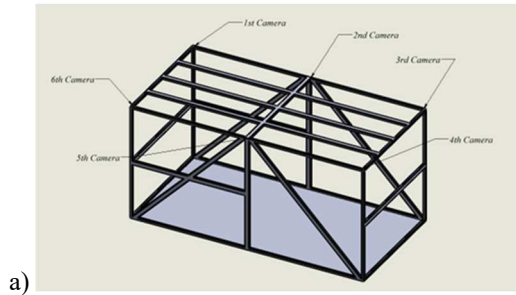


Fig. 5. Algorithm for the leader.

used to track the coordinates of the vehicles, and a PC with a router was used as a ground station. The enclosed workspace is covered with a safety net that protects operator(s) and along with 1cm rubber flooring (Fig. 7b), it prevents fatal crashes of UAV(s).



a)



b)

Fig. 7. a) 3D model and b) real enclosed workspace with a Safety net and rubber flooring with locations of 6 cameras.

The Ground station consists of the PC and router. The router receives data from the onboard modules of UAV(s) and UGV(s) and sends them to the PC. PC has installed QUARC software that only works with MATLAB & Simulink integrated development environment, therefore all the data that comes from the router and OptiTrack cameras are received by QUARC software. These data can be accessed by the Quanser toolbox from Simulink. All control system algorithms are located in Simulink, which allows developers to speed up the prototyping of controls and real-time HIL testing.

OptiTrack cameras are fixed on iron bars at a height of 2.5 meters from the ground to optimally cover the entire workspace (Fig. 8). Markers located on UAV(s) and UGV(s) bodies serve to accurately determine the required data. Collected data goes directly to the computer after which it can be used by Simulink.



Fig. 8. OptiTrack Camera.

QUANSER's Qdrone (Fig. 9) is used as an unmanned aerial vehicle with a pre-installed Intel Aero Computing Board to communicate with a ground station. Qdrone has five markers on the body, which allows the Optitrack to read the exact XYZ coordinates, pitch, yaw, and roll data. Qbot 2e is used as an unmanned ground vehicle with an onboard raspberry pi 3 model b+ module to communicate via Wi-Fi and process data from sensors. Qbot 2e also has 5 markers located on its body for the object's coordinates, pitch, roll, and yaw measurement purposes. For the Qbot 2e to be able to bypass obstacles, two ultrasonic sensors HC-SR04 were installed on the front of the bot on the right and left (Fig. 10).

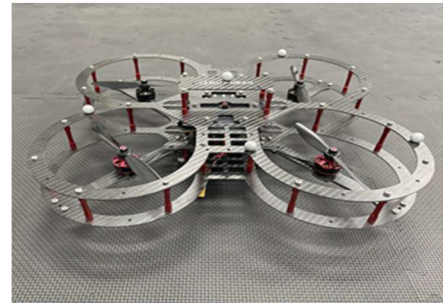


Fig. 9. QUANSER's QDrone.

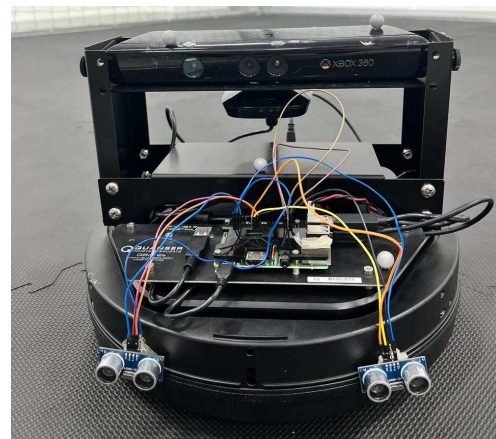


Fig. 10. QUANSER's QBot 2e with two ultrasonic sensors.

IV. RESULTS AND DISCUSSION

In order to test the algorithms for effectiveness in the different situations following 5 formations of obstacles were used. Fig. 11 shows the ideal trajectory of the algorithm and the obstacles that were placed.

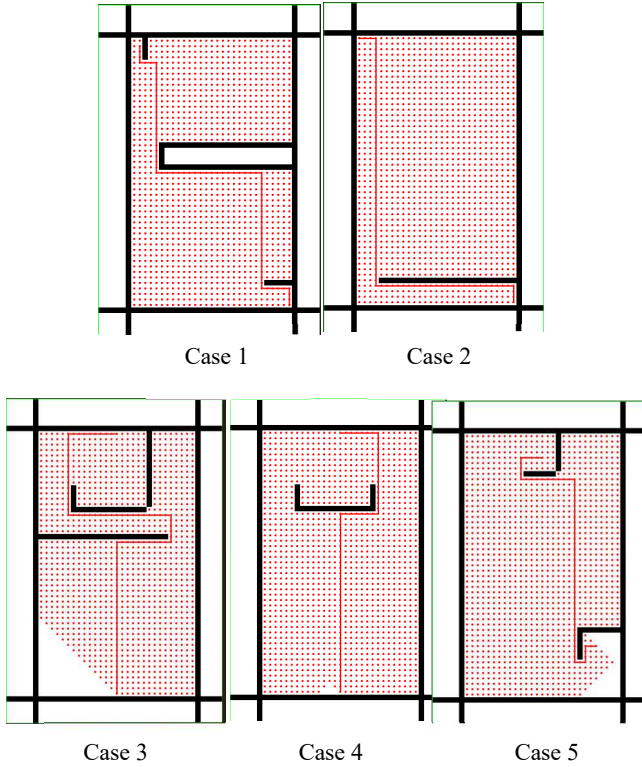


Fig. 11. Ideal paths generated by an algorithm to bypass five obstacle patterns.

TABLE I. TABLE TYPE STYLES

Case number	Number of obstacles that QBot had hit	Number of attempts to successfully return	Successful return to the QDrone
1	2	1	True
2	2	3	True
3	1	-	False
4	1	1	True
5	1	1	True

The first and fifth cases were successful because the obstacles were placed in such a way that the QBot had a low speed at the moment when the sensors fixed the obstacle. This provided good obstacle avoidance, and the sensors in front of the bot were not damaged after a collision with obstacles.

In the second and fourth cases, few obstacles were placed. However, the fourth case was easier to pass than the second. The fact is that in the fourth case, the obstacle is closer to the starting point of QBot, because of this, at the time of determining the presence of an obstacle, its speed is lower than in case 2. In the second case, QBot reached its maximum speed when moving towards QDrone, and did not have time to slow down before hitting the wall. Due to the

collision, the wires from the sensors were disconnected and they stopped working. Once the wires were reattached to the sensor, the QBot was finally able to get back to the QDrone on the third try.

In the third case, the robot failed to get to QDrone for all 6 attempts made. The reason was that during deceleration, QBot managed to fix a 5-decimeter thick wall directly in front of it, which blocked the existing 4-decimeter wide corridor, due to which the algorithms considered that there was no way to return to QDrone.

From the tested cases, several limitations of the current algorithm and system can be identified:

1. If QBot gains a high momentum, then it will record a wall in front of it several decimeters thick, which can sometimes lead to the inability to complete the task (as in case 3).
2. The path generated by the algorithm may not be the most optimal, since the presence of obstacles is recognized by determining them right in front of the sensors. As a result, QBot only bypasses obstacles and follows QBot, and does not go to the desired point by the shortest path.
3. The algorithm generates a path too close to obstacles, which sometimes causes unnecessary collisions and time management problems.

For a full experiment with waypoints and drone movement, a simpler set of obstacles was chosen. Two obstacles were placed inside the workspace (Fig. 12). One obstacle was placed right in front of the QBot at the start. The second one is placed between the first and second waypoint so that the follower avoids the obstacle during the passage of the predetermined trajectory.



Fig. 12. Location of obstacles.

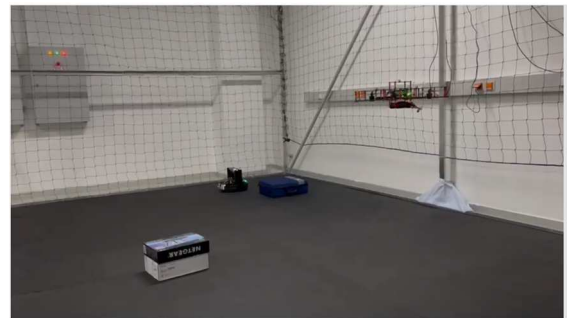


Fig. 13. Avoiding the first obstacle.

The trajectories of the leader and follower are shown in Fig. 14. As can be seen from the figure, both vehicles reached their assigned waypoints, which means that the leader did not start moving to the next waypoint until the follower was within a certain distance of it.

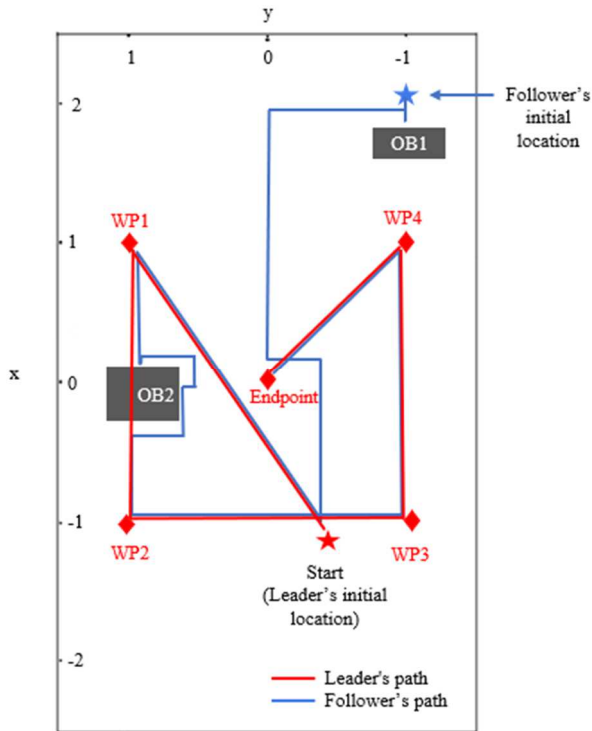


Fig. 14. The trajectory of the Vehicles (WP – waypoint, OB – obstacle).

Moreover, by analyzing the trajectory, it is clearly seen how the follower avoids obstacles. This also can be seen in Fig. 13 and Fig. 16. At the moment of bypassing obstacle No. 2, the QBot collided with the obstacle (Fig. 15). This happened due to the fact that the QBot accelerated from the previous stop and did not have time to slow down before the obstacle. To eliminate this drawback, it was decided to program the reading of sensors for long and short distances, however, it was found that the Raspberry Pi is not able to process such a large amount of information.

After the UGV collided with an obstacle, it bypassed it, but when it did the check, the sensors did not detect the obstacle. Although the obstacle slightly blocked the path of the UGV (Fig. 16), which resulted in the second collision. The fact is that due to the same limited computing power of the Raspberry Pi, it turned out to be impossible to use 3 sensors because as soon as the QBot connected to the computer, the script written for the sensors stopped working. For this reason, it was decided to place two sensors on two sides of the QBot, but at the same time that they both be directed forward. Which led to the formation of blind spots on two sides of the QBot.

Asad et al. designed a hurdle avoidance controller for QBot 2, using an Arduino Uno microcontroller [11]. This study also used 2 sonar sensors for obstacle detection. According to the results of the study, it can be concluded that the system built by Asad et al. more smoothly bypasses obstacles. This is because, in this study, the sensors read the distance of the obstacle and subsequently classify them as far, medium, and

near, which allows for a smoother response to triggers. Moreover, it is important to note that in this study, the onboard microcontroller did not communicate with the earth station, which did not create delays. This cannot be done in our study, because in addition to the task of avoiding obstacles, the goal of our study is also following the leader, which complicates the system.

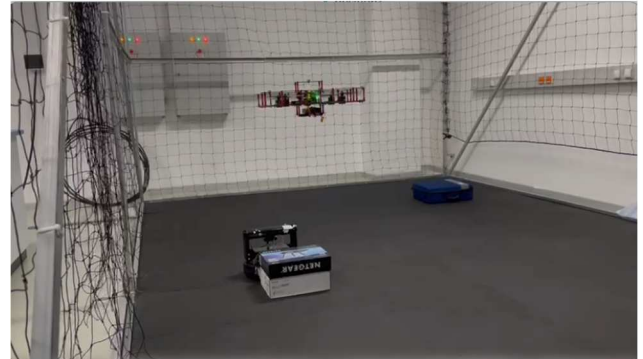


Fig. 15. Collision with an obstacle.

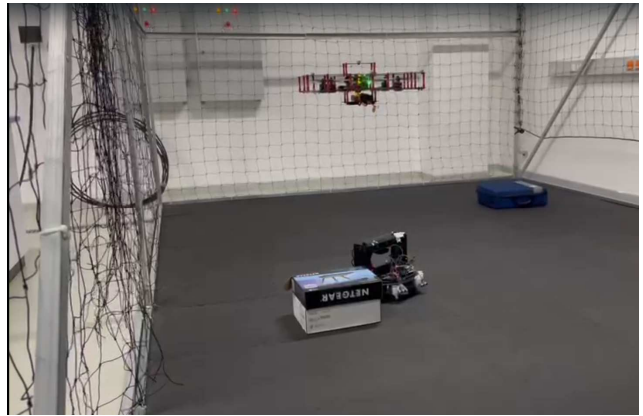


Fig. 16. Avoiding the second obstacle.

V. CONCLUSION

In conclusion, the results of the behavior of the vehicles showed that the system works well. Problems with the recalculation of the path to avoid obstacles were not found. Moreover, both vehicles pass the preassigned path with sufficient accuracy.

The algorithm that was tested in this research has its advantages and disadvantages that should be fixed. Future research work will be directed to making algorithms to generate the most efficient path and to ensure that path will not be too close to the obstacles. This improvement can be applied if the onboard processor directly sends the distance to obstacles to the ground station, rather than sending a signal when a critical distance is triggered. This was not possible at this stage of the study due to the power limitation of the embedded Raspberry Pi processor and the MEX file we used to simulate Matlab and Simulink. This MEX file provides error-free simulation by limiting the use of multiple modules such as transferring UDP/TCP files between QBot and the ground station over a wireless connection. Moreover, the reason why the onboard camera was not used in this experiment is also due to the limitations of the MEX file,

when removed, camera frames can be used with a trained neural network to recognize obstacles.

Acknowledgment

This research is funded by Nazarbayev University under Collaborative Research Program Grant No 11022021CRP1509.

REFERENCES

- [1] Y. Wu, Z. Zuo, Q. Han, Y. Wang, and H. Yang, "Formation Control of Wheeled Mobile Robots With Multiple Virtual Leaders Under Communication Failures," *IEEE Transactions on Control Systems Technology*, 2022.
- [2] S. Chang, Y., Wang, and Z. Zuo, "Formation Control for Wheeled Mobile Robots With Finite-Time Active Disturbance Rejection Control," in *2019 IEEE International Conference on Real-time Computing and Robotics (RCAR)*, pp. 818-822, August 2019.
- [3] A. Drak, M. Hejase, M. ElShorbagy, A. Wahyudie, and H. Noura, "Autonomous formation flight algorithm and platform for quadrotor uavs," *International Journal of Robotics and Mechatronics*, vol. 1, no. 4, pp. 124-132, 2014.
- [4] K. Gopakumar, and K. V. Shihabudheen, "Leader Follower Distributed Consensus Control of Heterogeneous Multi-agent System with Model Predictive Control," in *2021 International Symposium of Asian Control Association on Intelligent Robotics and Industrial Automation (IRIA)*, pp. 25-30, September 2021.
- [5] M. A. Kamel, X. Yu, and Y. Zhang, "Real-time optimal formation reconfiguration of multiple wheeled mobile robots based on particle swarm optimization," in *2016 12th World Congress on Intelligent Control and Automation (WCICA)*, pp. 703-708, June 2016.
- [6] M. A. Kamel, X. Yu, and Y. Zhang, "Fault-tolerant cooperative control design of multiple wheeled mobile robots," *IEEE Transactions on control systems technology*, vol. 26, no. 2, pp. 756-764, 2017.
- [7] M. A. Kamel, K. A. Ghamry, and Y. Zhang, "Real-time fault-tolerant cooperative control of multiple UAVs-UGVs in the presence of actuator faults," *Journal of Intelligent & Robotic Systems*, vol. 88, no. 2, pp. 469-480, 2017.
- [8] J. Luo, C. L. Liu, and F. Liu, "A leader-following formation control of multiple mobile robots with obstacle," in *2015 IEEE International Conference on Information and Automation*, pp. 2153-2158, August 2015.
- [9] K. A. Ghamry, Y. Dong, M. A. Kamel, and Y. Zhang, "Real-time autonomous take-off, tracking and landing of UAV on a moving UGV platform," in *2016 24th Mediterranean conference on control and automation (MED)*, pp. 1236-1241, June 2016.
- [10] NaturalPoint, Inc. DBA OptiTrack, "Motion Capture for Movement Sciences," OptiTrack, 2022. Retrieved July 18, 2022, from <https://OptiTrack.com/applications/movement-sciences/#:%7E:text=The%20most%20accurate%20measurement%20system,of%200.1%20mm%20or%20less>.
- [11] M. U. Asad, J. Gu, U. Farooq, R. Dey, V. E. Balas, and G. Abbas, "Intelligent Obstacle Avoidance Controller for QBot2," *IFAC-PapersOnLine*, vol. 55, no. 1, pp. 120-125, 2022.